

# App Dev-1 Project Report

## 1. Student Details

**Name:** Dheepak D

**Roll Number:** 23f2001453

**Email:** 23f2001453@ds.study.iitm.ac.in

**About Me:** I'm an IIT Madras BS student concurrently pursuing a full-time B.E. degree and an NCC cadet. My primary passion is learning new concepts and I find the greatest satisfaction and happiness when I can apply that knowledge by building a practical project. I apply the same in my full-time degree also.

---

## 2. Project Details

**Project Title:** Hospital Management System

### **Problem Statement:**

To design and build a web-based Hospital Management System (HMS) that allows **Admins, Doctors, and Patients** to efficiently manage and **track patient records, appointments, and treatment schedules according to their specific roles.**

### **Approach:**

The app was built using **Flask** as the backend framework with a modular structure. It allows **Admins, Doctors, and Patients** to securely access and manage their specific data, including **patient records, appointments, and treatment histories**, facilitating efficient hospital operations.

---

## 3. AI/LLM Declaration

I used an **AI/LLM** (Chat-GPT) to assist in generating **frontend CSS styling**, suggesting **database model logic**, and providing suggestions for repetitive codes for API endpoints.

The extent of AI/LLM usage is around **~15-20%**, limited to aesthetic suggestions, **~10-15%** for model logic and boilerplate code.

All final implementation logic, including the **database interactions (CRUD operations)**, **authentication/authorization setup**, and **integration of the three user roles (Admin, Doctor, Patient)**, were done manually.

---

## 4. Technologies and Frameworks Used

| Technology / Library | Purpose                                          |
|----------------------|--------------------------------------------------|
| Flask                | Core backend web framework                       |
| SQLAlchemy           | Object Relational Mapper for SQLite database     |
| Jinja2               | Template engine for rendering dynamic HTML pages |
| Bootstrap            | Frontend styling and responsive design           |
| Flask-Login          | User authentication and session management       |
| SQLite               | Lightweight local database for storing user data |

---

## 5. Database Schema / ER Diagram

The Hospital Management System (HMS) application utilizes seven main database models to manage user roles, appointments, medical records, and hospital resources:

1. **User:** Stores core user details and authentication info for all roles (**Admin, Doctor, Patient**), including the primary key **id**, username, email, password\_hash, and the essential **role** field.
2. **Department:** Stores hospital department information (e.g., Cardiology, Orthopedics), identified by its **id**, name, and description.
3. **Doctor:** Stores Doctor-specific details, linked via a one-to-one relationship to the **User** table (**user\_id**). Key fields include specialization and **department\_id**.
4. **Patient:** Stores Patient-specific details, linked via a one-to-one relationship to the **User** table (**user\_id**). It includes fields like emergency\_contact.
5. **DoctorAvailability:** Stores specific time slots and schedules when a doctor is available. It links to the **Doctor** via **doctor\_id** and includes fields like date, start\_time, end\_time, and slot\_capacity.
6. **Appointment:** Stores scheduled appointments, which link a **Patient** (**patient\_id**), a **Doctor** (**doctor\_id**), and a specific **DoctorAvailability** slot (**availability\_id**), along with the date, time, and status.
7. **Treatment:** Stores the medical record or notes for a specific appointment, linked via a one-to-one relationship to the **Appointment** table (**appointment\_id**). Fields include diagnosis, prescription, and treatment\_notes.

## Relationships:

- **One-to-One:** User → Doctor
- **One-to-One:** User → Patient
- **One-to-One:** Appointment → Treatment
- **One-to-Many:** Department → Doctor
- **One-to-Many:** Doctor → DoctorAvailability
- **One-to-Many:** Doctor → Appointment
- **One-to-Many:** Patient → Appointment
- **One-to-Many:** DoctorAvailability → Appointment



## 6. API Resource Endpoints

### 1. Authentication & Session Management

| Endpoint  | Method | Description                                                          |
|-----------|--------|----------------------------------------------------------------------|
| /login    | GET    | Displays the login page.                                             |
| /login    | POST   | Authenticates the user and creates a secure session.                 |
| /logout   | GET    | Ends the user's session and redirects to the homepage.               |
| /register | GET    | Displays the multi-role registration form (initially for patients).  |
| /register | POST   | Creates a new User account and associated Doctor or Patient profile. |

### 2. Dashboard Endpoints

| Endpoint           | Method | Description                                                             |
|--------------------|--------|-------------------------------------------------------------------------|
| /                  | GET    | Root path—redirects the logged-in user to their specific dashboard.     |
| /admin/dashboard   | GET    | Displays the Admin overview with system-wide metrics.                   |
| /doctor/dashboard  | GET    | Displays the Doctor's schedule and pending appointments.                |
| /doctor/dashboard  | POST   | Handles updating appointment statuses (e.g., Confirm, Treat, Complete). |
| /patient/dashboard | GET    | Displays the Patient's overview and list of doctors.                    |

### 3. Admin - User Management

| Endpoint                   | Method   | Description                                          |
|----------------------------|----------|------------------------------------------------------|
| /admin/patient/add         | GET/POST | Displays the form and creates a new Patient account. |
| /admin/patient/edit/<id>   | GET/POST | Displays the form and updates Patient information.   |
| /admin/patient/delete/<id> | POST     | Deletes the specified Patient account.               |
| /admin/doctor/add          | GET/POST | Displays the form and creates a new Doctor account.  |
| /admin/doctor/edit/<id>    | GET/POST | Displays the form and updates Doctor information.    |
| /admin/doctor/delete/<id>  | POST     | Deletes the specified Doctor account.                |

### 4. Doctor - Availability Management

| Endpoint                         | Method   | Description                                          |
|----------------------------------|----------|------------------------------------------------------|
| /doctor/availability             | GET/POST | Views the schedule and Adds a new availability slot. |
| /doctor/availability/edit/<id>   | GET/POST | Displays the form and updates an availability slot.  |
| /doctor/availability/delete/<id> | POST     | Deletes the specified availability slot.             |

## 5. Patient & Doctor - Appointment Management

| Endpoint                         | Method | Description                                        |
|----------------------------------|--------|----------------------------------------------------|
| /patient/appointments            | GET    | Views all appointments booked by the Patient.      |
| /doctor/appointments             | GET    | Views all appointments assigned to the Doctor.     |
| /doctor/<id>/book                | POST   | Books a new appointment with the specified doctor. |
| /patient/appointment/<id>/cancel | POST   | Cancels a specific booked appointment.             |

## 6. Treatment & Medical Records

| Endpoint                | Method   | Description                                                                   |
|-------------------------|----------|-------------------------------------------------------------------------------|
| /doctor/treat/<id>      | GET/POST | Displays the form and Saves the diagnosis/prescription for an appointment ID. |
| /patient/treatment/<id> | GET      | Allows the patient to view their medical record (treatment details).          |

---

## 7. Architecture and Features

### Architecture Overview:

- app.py** – main Flask application entry point
- /models** – database models using SQLAlchemy
- /routes** – Flask Blueprints for user and activity routes
- /templates** – Jinja2 HTML templates
- /static** – CSS, and profile image file

---

## 8. Video Presentation

Drive Link: [Project Video](#)